

Hotel Management App: Issues and Improvement Report

Summary

The application is functional in core areas, but several usability, validation, and role-access improvements are needed before it is suitable for non-technical hotel guests and staff.

Critical Issue: Registration Validation Crashes the App

Observed behaviour: Registering with an invalid password, for example three characters, returns API status `422 Unprocessable Entity` and crashes the mobile app.

Visible error:

```
Objects are not valid as a React child... keys {type, loc, msg,
input, ctx}
```

Root cause: The backend returns validation errors as a list of objects. The frontend stores that list in its `error` state and attempts to render it directly inside a text element.

Required fix:

- Convert backend validation errors into a plain, user-friendly message before rendering.
- Show a clear inline message, for example: "Password must be at least 8 characters."
- Add client-side validation before sending the registration request.
- Do not allow any API error object or array to be rendered directly in the UI.

Acceptance criteria:

- Invalid passwords, emails, and required fields never crash the app.
- The user sees one clear correction message.
- Registration remains usable immediately after an error.

Date Input and Booking Validation

Issue: Date fields need safer formatting and validation. Invalid formats, past dates, or checkout dates before check-in must not cause an app failure or confusing results.

Required fix:

- Use a native date picker instead of unrestricted text entry where possible.
- Display dates consistently, for example 01 Sep 2026.
- Send dates to the API in ISO format: YYYY-MM-DD.
- Reject past check-in dates.
- Reject checkout dates that are on or before check-in.
- Show a simple alert explaining what the user needs to correct.
- Preserve the entered booking form values after validation fails.

Acceptance criteria:

- Invalid date input never crashes the app.
- Guests receive clear guidance before making an API request.
- The backend continues validating all date rules as the final safeguard.

UI/UX Improvements for Guests, Front Desk, and Admin

Goal: Make all screens easier to use for non-technical guests and operational staff.

Guest experience

- Use plain labels such as “Choose dates,” “Find rooms,” “Confirm booking,” and “My stays.”
- Reduce technical terminology and unnecessary options.
- Use larger tap targets, clear room cards, prominent prices, and a visible booking summary.
- Add loading, empty, success, and error states to every guest flow.
- Show confirmation details immediately after booking.

Front Desk experience

- Prioritize today’s arrivals, departures, occupied rooms, and rooms needing cleaning.
- Use colour-coded room status labels with text, not colour alone.
- Place Check-in, Check-out, and Room Status actions close to the guest/room information.
- Add confirmation before irreversible operational actions.

Admin experience

- Keep KPI cards simple and explain their meaning.
- Make inventory, staff, reports, and audit logs easy to scan and filter.

- Add search, date range filters, and clear empty states.

Chat Keyboard Experience

Issue: When a guest taps the chat input, the keyboard can hide the latest messages or input area.

Required fix:

- Wrap the chat screen in `KeyboardAvoidingView`.
- Use a keyboard-aware scroll container.
- Automatically scroll to the latest message when the keyboard opens or a message is sent.
- Keep the message input and Send button visible above the keyboard.
- Allow tapping outside the keyboard without losing conversation context.

Booking Invoice Requirement

Missing feature: No invoice generation feature currently exists.

Required behaviour: After a booking is confirmed, generate an invoice that can be viewed by:

- The Guest who created the booking.
- Front Desk staff.
- Admin users.

Invoice should include:

- Hotel name and contact details.
- Booking reference.
- Guest name.
- Room number/type.
- Check-in and check-out dates.
- Number of nights.
- Nightly rate, subtotal, taxes/fees if applicable, and total amount.
- Booking status and issue date.

Recommendation:

- Create a protected invoice endpoint, for example `GET /api/bookings/{id}/invoice`.
- Enforce access: booking guest, Front Desk, and Admin only.
- Start with an in-app invoice screen and printable HTML/PDF export later.

Front Desk Audit and Reports Access

Current behaviour: Audit logs and reports are Admin-only. The existing audit-log API requires the Admin role, and Front Desk has no reports/audit screen.

Requested change:

- Give Front Desk read-only access to operational audit entries relevant to their work.
- Give Front Desk read-only access to booking and room-operation reports.
- Do not allow Front Desk to see sensitive Admin actions, staff-management events, configuration changes, or system secrets.

Recommended Front Desk audit fields:

- Timestamp.
- Staff member.
- Action type.
- Booking reference or room number.
- Guest name where appropriate.
- Before/after room status.
- Action details.

Recommended permissions:

- Guest: view only their own booking invoice.
- Front Desk: view operational invoices, operational reports, and limited audit records.
- Admin: full reports, full audit logs, inventory, staff, and policy administration.

Priority Order

1. Fix registration error rendering and client-side validation.
2. Improve date validation and alerts.
3. Add booking invoice generation and role-based viewing.
4. Improve chat keyboard handling and user experience.
5. Improve UI/UX across all roles.
6. Add Front Desk read-only audit/report access.